netmaster：基于eBPF的网络延迟与故障诊断工具

The 4th eBPF Developer Conference (eBPFDC 2026)

陕西 · 西安　　2026年4月18日

陕西 · 西安　2026年4月18日

项目背景

云原生时代，网络可观测性成为基础设施的核心能力

架构复杂度飙升

云原生 / 微服务架构下
网络调用链呈指数级增长
服务间通信拓扑动态变化

性能故障频发

网络性能问题已成为
业务故障的首要原因
直接影响用户体验

传统工具力不从心

tcpdump / sar / ss 等工具
定位粒度粗、开销大
无法满足精细化诊断需求

可观测性迫切需求

企业亟需覆盖全链路的
内核级网络诊断能力
支撑大规模生产环境运维

网络复杂度↑　故障定位难度↑　传统工具局限↑　可观测需求↑

网络故障排查的现实痛点

传统方案在云原生场景下面临四大核心挑战

定位难
延迟发生在哪一层？协议栈 / 驱动 / 硬件？

关联难
应用层超时与内核层丢包之间缺乏因果链

开销大
tcpdump 在高吞吐场景下 CPU/内存开销不可接受

侵入性强
传统方案需要修改内核代码或重启业务进程

eBPF 技术概述

netmaster 的核心底层技术基石

eBPF — 在内核中安全运行沙箱程序

定义

Linux 内核中的虚拟机（字节码引擎），可在不修改内核源码、不加载内核模块的前提下，安全地在内核态执行用户自定义逻辑

核心机制

BPF 验证器在加载时静态分析程序，确保无死循环、无越界访问、无非法内存操作，通过后 JIT 编译为原生机器码执行

数据交互

通过 eBPF Map（ringbuf / hash / array 等）实现内核态与用户态之间的高效数据传递，ringbuf 支持零拷贝

可移植性

BTF（类型格式）+ CO-RE（Compile Once, Run Everywhere）实现编译一次、跨内核版本运行

Hook 点类型

kprobe / kretprobe — 内核函数探测
tracepoint — 内核静态跟踪点
fentry / fexit — 更高效的函数入口/出口

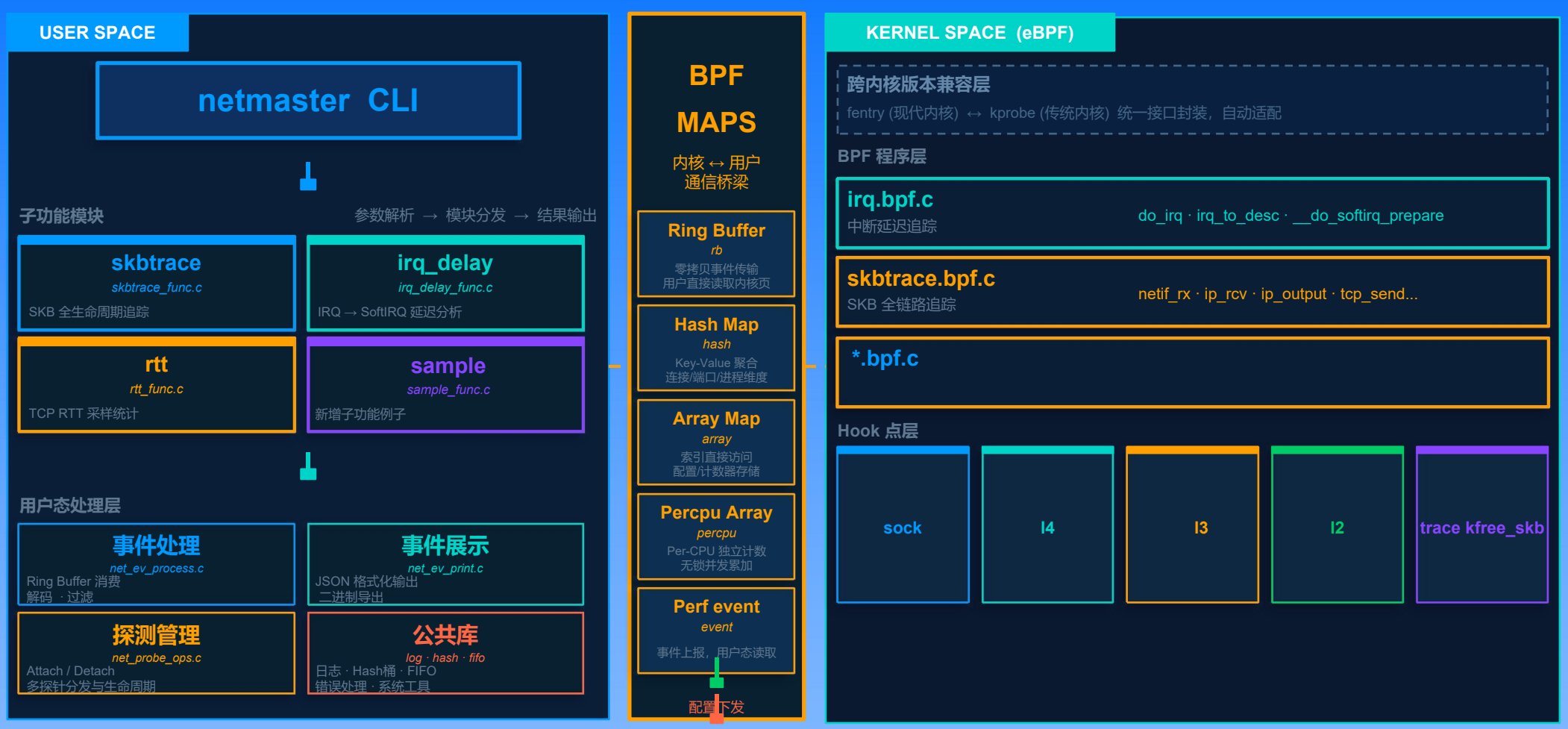
eBPF Map 类型

ringbuf — 高效环形缓冲区（推荐）
perf_event — 传统事件输出
hash — 键值对统计表
array — 固定大小数组
per-CPU 变体 — 避免锁竞争

安全保障

BPF 验证器静态检查
指令数上限防止死循环
map max_entries 内存上限
CO-RE 避免硬编码偏移
无法访问任意内核内存

netmaster 整体架构



内核网络栈收发全景

发送路径 (TX)：头部从无到有，逐层封装

用户态
- 应用数据

无任何协议头，只有原始数据

传输层
- TCP/UDP头
- 应用数据

封装传输层头部：源端口、目的端口、序列号、校验和、窗口大小等

网络层
- IP头
- TCP/UDP头
- 应用数据

封装 IP 头部：源IP、目的IP、TTL、协议号、IP校验和

链路层
- MAC头
- IP头
- TCP头
- 数据
- FCS

封装以太网头+FCS：源MAC、目的MAC、EtherType，帧完整离开网卡

物理层
- 完整帧

所有头部就绪，通过网线/光纤发出

接收路径 (RX)：头部从有到无，逐层剥离

物理层
- 完整帧

完整帧到达网卡，所有头部均存在

链路层
- IP头
- TCP/UDP头
- 应用数据

剥离以太网头+FCS：校验目的MAC → 协议分发至上层

网络层
- TCP/UDP头
- 应用数据

剥离 IP 头部：校验和验证、TTL检查、分片重组

传输层
- 应用数据

剥离 TCP/UDP 头部：序列号校验、放入接收队列、唤醒进程

用户态
- 应用数据

仅剩 Payload，头部全部剥离完成

收发路径、不同层级携带信息不一致

全链路事件关联追踪

收发双向 · 分层 hook 点关联 · sock → skb 逐层 Key 映射

层级	▼ 发送 Send	▲ 接收 Recv	Key
用户态 User Space	**send() / write()** 通过 fd 找到 sock，触发内核写入	**recv() / read()** 从 sock 拷贝数据到用户缓冲区	**fd**
Socket 层 BSD Socket	**tcp_sendmsg** sock 构造 sk_buff，挂入发送队列	**tcp_recvmsg** 从 sk_buff 拷贝数据到 sock	**sock**
TCP 层 Transport	**tcp_transmit_skb** 分段、拥塞控制、序列号分配	**tcp_v4_do_rcv** 校验和、ACK 生成、乱序重组	**skb**
IP 层 Network	**ip_queue_xmit** 路由查找、TTL、分片	**ip_rcv** 重组、netfilter、路由判断	**skb**
驱动/软中断 Driver/SoftIRQ	**__qdisc_run → xmit** Qdisc 排队调度 → 硬件发送	**napi_poll → netif_rx** 硬中断触发软中断 → NAPI 收包	**skb**
硬中断 Hard IRQ	**DMA 完成中断** 网卡发送完成，释放 ring buffer	**RX 中断** 网卡收到帧，触发硬中断通知 CPU	—

同一 skb 指针跨层哈希关联 · 各阶段耗时 = 当前时间戳 − 上一时间戳 · Key 链: sock → skb（发）/ skb → sock （收）

工具性能优化

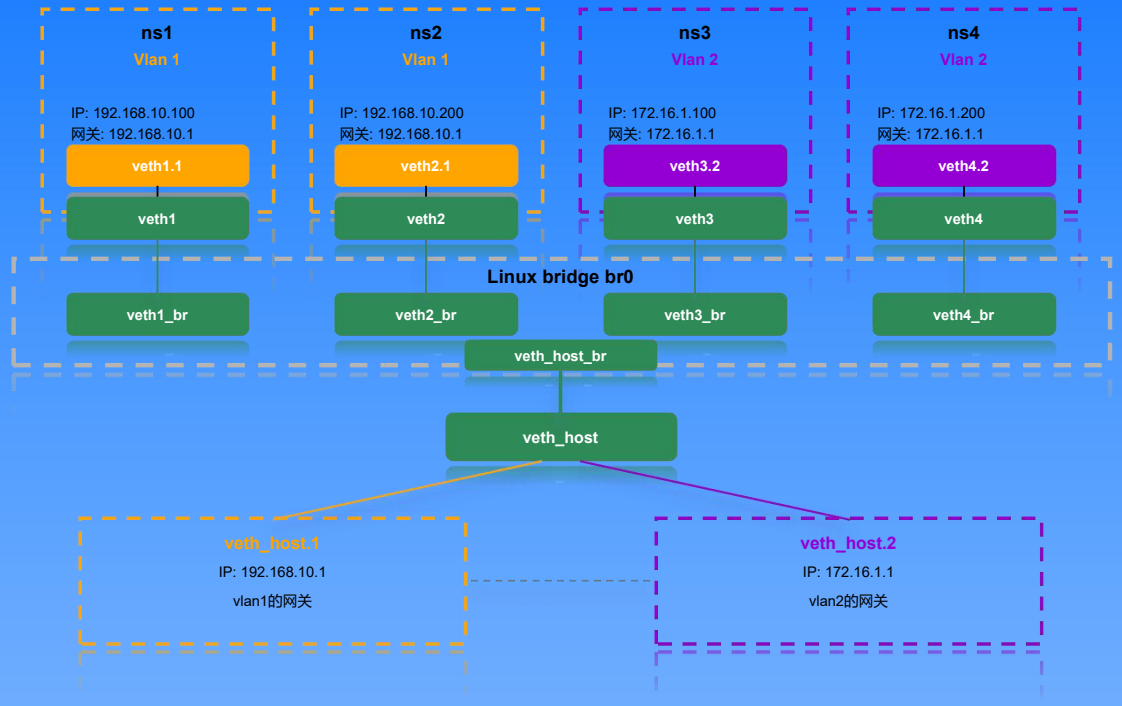
- **Hook 点按功能分组**
  命令行可按组控制是否attach
- **每个 Hook 点独立配置**
  每个hook都可单独配置是否attach
- **Ring Buffer 高效传输**
  零拷贝批量消费
- **fentry / fexit 支持**
  相比kprobe更加高效
- **支持二进制日志记录**
  比字符输出到文件体积减少 80%+，解析效率高
- **Per-SKB 全链路记录**
  唯一 Trace ID，内核侧关联，无需用户态拼接
- **精简模式**
  精简模式，每层只保留一个HOOK点，只做延迟计算

报文流转、延迟情况可视化

```
-----------------------------------------------------------------------------------------------------------------
SKB:0xffff99b624eec500,CPU:3 |ICMP: 192.168.10.100->172.16.1.100 ping request, seq: 23, id: 20560
-----------------------------------------------------------------------------------------------------------------
[81082.434711][0.000000]                 ip_output |
[81082.434722][0.000011]          ip_finish_output | <veth1.1> ICMP: 192.168.10.100->172.16.1.100 ping request, seq: 23, id: 20560
[81082.434724][0.000002]         ip_finish_output2 |
[81082.434727][0.000003]          __dev_queue_xmit |
[81082.434730][0.000003]       dev_hard_start_xmit |
[81082.434731][0.000001]          __dev_queue_xmit | <veth1> ICMP: 192.168.10.100->172.16.1.100 ping request, seq: 23, id: 20560
[81082.434732][0.000001]       dev_hard_start_xmit |
[81082.434735][0.000003]        enqueue_to_backlog | <veth1_br> ICMP: 192.168.10.100->172.16.1.100 ping request, seq: 23, id: 20560
[81082.434739][0.000004]  __netif_receive_skb_core |
[81082.434741][0.000002]         netif_receive_skb |
[81082.434745][0.000004]         br_nf_pre_routing |
[81082.434750][0.000005]              nf_hook_slow | <veth_host_br> ICMP: 192.168.10.100->172.16.1.100 ping request, seq: 23, id: 20560  *b
[81082.434753][0.000003]          br_nf_forward_ip |
[81082.434756][0.000003]         br_nf_forward_arp |
[81082.434758][0.000002]              nf_hook_slow | *bridge in chain: POST_ROUTING*
[81082.434760][0.000002]        br_nf_post_routing |
[81082.434762][0.000002]          __dev_queue_xmit |
[81082.434764][0.000002]       dev_hard_start_xmit |
[81082.434765][0.000001]        enqueue_to_backlog | <veth_host> ICMP: 192.168.10.100->172.16.1.100 ping request, seq: 23, id: 20560
[81082.434775][0.000010]  __netif_receive_skb_core |
[81082.434776][0.000001]         netif_receive_skb |
[81082.434778][0.000002]                   ip_rcv | <veth_host.1> ICMP: 192.168.10.100->172.16.1.100 ping request, seq: 23, id: 20560
[81082.434780][0.000002]              nf_hook_slow | *ipv4 in chain: PRE_ROUTING*
```

```
从veth1.1网卡ping veth3.2网卡，ping请求从veth1.1发出经流的网卡路径：
veth1.1 -> veth1 -> veth1_br -> veth_host_br -> veth_host -> veth_host.1 -> veth_host.2 -> veth_host -> veth_host_br
                                                                                                        |
                                                                                veth3.2 <- veth3 <- veth3_br  <

ping包回复从veth3.2发出，最终到达veth1.1网卡：
veth3.2 -> veth3 -> veth3_br -> veth_host_br -> veth_host -> veth_host.2 -> veth_host.1 -> veth_host -> veth_host_br
                                                                                                        |
                                                                                veth1.1 <- veth1 <- veth1_br  <
```



为了更加轻量级进行关键信息的监测，工具支持精简模式下的问题定界分析：

发送路径 (TX)：头部从无到有，逐层封装

用户态

应用数据

无任何协议头，只有原始数据

传输层

TCP/UDP头 | 应用数据

封装传输层头部：源端口、目的端口、序列号、校验和、窗口大小等

网络层

IP头 | TCP/UDP头 | 应用数据

封装 IP 头部：源IP、目的IP、TTL、协议号、IP校验和

链路层

MAC头 | IP头 | TCP头 | 数据 | FCS

封装以太网头+FCS：源MAC、目的MAC、EtherType，帧完整离开网卡

物理层

完整帧

所有头部就绪，通过网线/光纤发出

```
[X] sk: 0xffff8f4c26170980, skb: 0xffff8f4c14002ae8, pid: 0, comm: swapper/2, clone: 1, deepth_flag: 1
TCP 192.168.0.108:22 => 192.168.0.116:55432, seq: 226916326, ack: 3091086932 flags: AP pkt_len: 378
[latency]:
cork_enq_delay: 1446ns
cork_outq_delay: 1750ns
ip_queue_xmit_delay: 5724ns
net_dev_queue_delay: 56396061ns
dev_hard_start_xmit_delay: 56427103ns
func chain:
call stack:
        00 [<ffffffffa93c3691>] __tcp_transmit_skb+0x1
        01 [<ffffffffa93c5dc1>] tcp_write_xmit+0x2c1
        02 [<ffffffffa93c67b9>] __tcp_push_pending_frames+0x39
        03 [<ffffffffa93a9ab0>] tcp_sendmsg_locked+0x7a0
        04 [<ffffffffa93aa5af>] tcp_sendmsg+0x2f
        05 [<ffffffffa927e591>] sock_write_iter+0x181
        06 [<ffffffffa88017b4>] vfs_write+0x3c4
        07 [<ffffffffa8801a9f>] ksys_write+0xbf
        08 [<ffffffffa95a088e>] do_syscall_64+0x7e
        09 [<ffffffffa800012f>] entry_SYSCALL_64_after_hwframe+0x76
```

其它支持的一些功能

覆盖丢包诊断、报文Dump、内核故障、软中断延迟、队列积压等关键场景

丢包监测功能

支持头部信息、数据区 dump 功能，适应无法解析报文的场景

关键内核函数返回值监测

监测内核协议栈关键函数处理返回值，用于内核协议栈处理故障诊断

NO_SOCK 根因分析

自动识别和分析无 socket 关联报文的根因，快速定位连接异常

网卡接收软中断延迟监测

精确追踪软中断处理延迟，定位网络接收路径中的性能瓶颈

sock 队列积压监测

实时监控 socket 队列积压情况，预警拥塞导致的延迟问题

实际案例

案例一
问题：某业务出现延迟

解决：通过耗时分析，定位到 cork 队列堆积

案例二
问题：客户反馈出现网络延迟

解决：通过软中断延迟功能，定位到软中断

案例三
问题：vlan组播丢包问题分析

解决：定位到丢包位置

未来规划

服务化
后台常驻，以精简模式采集信息，以这些信息触发事件，进行自动的信息采集

数据可视化
支持Grafana等前端展示

跨主机信息关联
支持跨主机的相同请求链延迟关联